

- ```
>>> d = {1: 'David' , 2: 'Goliath', 3: 'Judith' , 4: 'Saul'}
```

## Accessing Data

An item in the dictionary can be accessed by providing the corresponding dictionary key. If the key is not found, the operation raises a `KeyError`. A similar operation can be performed using the `get()` method.

- ```
>>> d[1]
```
- ```
'David'
```
- ```
>>> d[5]
```
- ```
Traceback (most recent call last):
```
- ```
File "<stdin>", line 1, in <module>
```
- ```
KeyError: 5
```
- ```
>>> d.get(2)
```
- ```
'Goliath'
```

With looping techniques, we can extract `(key, value)` at the same time using the `items()` method.

- ```
>>> for key, value in d.items():
```
- ```
... print(key, value)
```
- ```
...
```
- ```
1 David
```
- ```
2 Goliath
```
- ```
3 Judith
```
- ```
4 Saul
```

Manipulating Data

While dealing with dictionaries we might often need to add, modify or delete data. A dictionary provides various methods and techniques for each kind.

- In order to assign a value to a key, we can directly put the value against the key. If the key was not present before the assignment, a new `(key, value)` pair is added to the `dict`, if the `key` existed previously the value associated with the `key` is overwritten. Updating the dictionary with multiple `(key, value)` pairs can be done using `update()` function, which accepts another dictionary or an iterable of `(key, value)` pairs.